Constrained Sequence Alignment: A Dedicated Version and Its Applications

Yun-Sheng Chung*and Chuan Yi Tang
Department of Computer Science
National Tsing Hua University, Hsinchu, Taiwan
g914342@oz.nthu.edu.tw, cytang@cs.nthu.edu.tw

Abstract

In this paper, we study a problem that arises naturally in biological applications. Given two sequences, along with a sequence of patterns, we want to align the two sequences such that the specified patterns are aligned together. This is the constrained sequence alignment problem and is defined in [14]. The multiple sequence version is called CMSA. In this paper, we focus on the pairwise version where one of the sequences is annotated with the correct occurrence positions of the patterns. This version solves many applications that one would use constrained alignment to solve, and the algorithm proposed here is more efficient than that for the original version. In addition, we propose an improved approximation algorithm for CMSA. Our algorithm improves that proposed in [3] from $O(Ck^2mn^2)$ time and $O(k^2mn^2)$ space to $O(Ck^2n^2)$ time and $O(kn)$ space, where k is the number of sequences and C is the maximum number of valid "constrained lists" (defined later).

1 Introduction

Sequence alignment is essential in computational biology. Classical methods consider the alignment of characters on the sequences. This problem has been extensively studied in the past (see, e.g., [10]). There are some properties that can be deduces directly from the characters, e.g. hydrophilicity and polarity of amino acids. Methods dealing with simple characters can naturally be used to treat this kind of properties. In [11], the amino acid sequences are transformed into these volumes so that FFT can be utilized to accelerate the alignment procedure. As biological knowledge, predictions and hypotheses grow rapidly, it turns out to be important and desirable to incorporate richer biological information into alignment procedures, instead of merely comparing characters or corresponding quantities. This motivates the study of annotated sequence alignment (analysis) [4], since such information often can be represented as some kind of annotations on the sequences. In the past, the study of annotated sequence alignment focuses on the annotation of RNA structure information, e.g., [12, 9] (each predicted hydrogen bond is represented as an arc linking two characters). In addition to arc annotations, Evans [4] also studied color annotations on sequences and applied them in database searches. Taylor used a different approach [15] for annotating and comparing motifs.

Tang et al. [14] studied a problem called constrained multiple sequence alignment (CMSA). This problem can also be recognized as one kind of annotated sequence alignment. In [14], a constraint is a sequence of residues that is considered necessary for a protein family. They considered RNases for an example. RNases contain three conserved His12 (H), Lys41 (K) and His119 (H) active site residues as their major structural feature [14]. Thus if we are aligning RNase sequences, it is desirable that each of these conserved residues appears in the same columns in the resulting alignment If standard alignment is used, those active site residues are not guaranteed to align together, which is not reasonable if we are given the knowledge. Furthermore, the order of "H, K, H" should be preserved. In the terminology in [14], "H, K, H" is the constraint to be satisfied in this example.

In a follow-up work by Chin et al. [3], a more efficient algorithm for the pair-wise constrained alignment is proposed. Their algorithm runs in $O(mn^2)$ time and space. For CMSA, they applied the center-star technique in Gusfield's well known 2-approximation algorithm for (standard) multiple sequence alignment [6] and get a 2-approximation algorithm.

*To whom correspondence should be addressed.

Tsai et al. [16] generalized the definition of patterns, from single residues to substrings. Their algorithm is equally efficient as that in [3]. They applied their method to detect if SARS-TW1 virus forms in some structural elements involving in RNA replications.

In this paper, we discuss the case when the positions of occurrences of the given patterns on one of the sequences are known. For the other sequence, we do not know at the beginning whether these patterns exist, or where they are. This is often the case in applications but is not discussed in [14, 3, 16]. Biologists are generally able to annotate important regions on a protein sequence known to have some function. Given a sequence whose function is to be predicted, we can align it with the annotated sequence to see if it can have the same function as the annotated sequence. Important regions can have different degrees: necessary, relevant, irrelevant, etc. For example, it has been shown that in Dihydropyrimidinases, there is a sequence of six residues responsible for the binding of metal [1]. In general, if a reasonable multiple sequence alignment is available for a protein family, we can extract conserved regions from the alignment, and the degree of conservation (e.g. perfectly conserved, strongly conserved, etc.) can be seen as an indication of importance. This suggests that an effective tool should be able to consider both necessary and relevant regions. Previous results either focuses only on necessary regions [14, 3, 16, 15], or only on relevant regions [4]. The algorithm presented in this paper deals with necessary regions, with additional constraints on ranges between patterns. In fact, the incorporation of regions of less important regions into our algorithm is straightforward and we discuss this later. Our algorithm runs in $O(n^2)$ time and $O(n)$ space. The result applies to the general cost measure as in [14, 3, 16].

The patterns we consider in this paper admits gaps and degenerate forms in them, which are not considered in [14, 3, 16, 15]. The method in [4] does not guarantee that all necessary regions are aligned together.

The constrained sequence alignment problem is in some sense similar to the color annotated sequence comparison problem [4]. But the color annotation (layered-string version) on a string forms in a tree structure and by definition any two color blocks in the same level can not overlap. This property enables one to solve this problem recursively by considering each colored substring a "hyper-character". But in our problem, this property no longer exists, since a pattern can be of length more than one and can occur in any position.

The remainder of this paper is organized as follows. In Sec. 2 we give some definitions. Section 3 gives the algorithm for the one-annotated version. Section 4 devotes to the improvement of the approximation algorithm in [3]. Finally, concluding remarks are given in Sec. 5.

2 Preliminaries

Let Σ be the alphabet where the characters of sequences are drawn. For a sequence X, let $X(i)$ be the ith character on X, and denote the string $X(i)X(i+1)\dots X(j)$ as $X[i..j]$. Denote the length of X as $|X|$.

An alignment of strings $S_1, \dots, S_k$ is a set of strings $(S'_1, \dots, S'_k)$, where S'_i is obtained by inserting some spaces "$-$" into S_i, and $|S'_i| = |S'_j|$ for all i, j. A pairwise alignment of two strings is a special case when $k = 2$. For convenience, define $\#spaces(c; S'_i)$ to be the number of space characters "$-$" in S'_i at or before position c. Also define $\#spaces(\mathcal{I}; S'_i)$ be the number of $-$'s in the interval $\mathcal{I}$ in S'_i.

Let $\delta : (\Sigma \cup \{-\})^2 \to \mathbf{R}$ be the cost function. Let (S'_1, S'_2) be an alignment. The score of this alignment is given by $\sum_l \delta(S'_1(l), S'_2(l))$.

Now we define some terms used in this paper.

Definition 1 *A pattern P is a language (set of strings) $P(1)P(2)\cdots P(|P|)$, where $P(l) \subset \Sigma$ for $1 \le l \le |P|$. We say that P occurs at position i in string X if $X[i+l-1] \in P(l)$ for $1 \le l \le |P|$. If P occurs in X at positions $i_1 < \dots < i_c$, then i_x is denoted as $start_x(X; P)$. Let $end_x(X; P) = i_x + |P| - 1$. We also say that P occurs at interval $[start_x(X; P), end_x(X; P)]$.*

As can be seen, the patterns are defined as languages instead of single residues. This definition is beneficial for dealing with degenerate forms, e.g., motifs that are represented in IUPAC code [5]. For a pattern P_l, define $cost(P_l)$ to be the cost of matching two occurrences of P_l. Later we refer to this kind of pattern as type-1 pattern.

We now give another type of pattern to be considered in this paper. This is useful to allow gaps in pattern occurrences.

Definition 2 *A pattern P is a string $P(1)P(2)\cdots P(|P|)$, where $P(l) \in \Sigma$ for*

$1 \le l \le |P|$. *We say that* P *occurs at interval* $[i, j]$ *in string* X *if* $d_E(X[i..j], P) \le t$, *where* t *is a threshold value and* d_E *is the weighted edit distance between two strings. Suppose that* P *occurs at intervals* $[i_1, j_1], [i_2, j_2], \ldots, [i_c, j_c]$, *where* $i_1 \le i_2 \le \cdots \le i_c$. *Then we denote* i_x *as* $start_x(X; P)$ *and* j_x *as* $end_x(X; P)$.

We will refer to this kind of pattern as type-2 pattern, for distinction. The definition for patterns adopted in [16] uses Hamming distance and does not allow gaps in occurrences. It is a special case of the definition given here.

Definition 3 *A motif constraint is a sequence (rather than merely a set) of patterns* $P_1, \ldots, P_m$. *We use* $\mathcal{P} = \langle P_1, \ldots, P_m \rangle$ *to denote a motif constraint. Also let* $\mathcal{P}_l = \langle P_1, \ldots, P_l \rangle$.

Definition 4 *An alignment* $S'_1, \ldots, S'_k$ *satisfies motif constraint* $P_1, \ldots, P_m$ *(either type-1 or type-2) if there exists intervals* $[c_1, c'_1], \ldots, [c_m, c'_m]$ *where* $c_1 < \cdots < c_m$ *such that for all* $1 \le i \le k$ *and all* $1 \le l \le m$, P_l *occurs at interval* $[c_l, c'_l]$ *in* S_i *and all* $[c_l, c'_l]$*'s are disjoint (referred to as the non-overlapping condition). When* $k = 2$ *and if* (S'_1, S'_2) *is optimal, the interval* $[c_l - \#spaces(c_l; S'_i), c_l - \#spaces(c_l; S'_i)]$ *is said to be a "correct" interval of occurrence of* P_l *in* S_i.

In this paper, the following problem is studied.

One-annotated constrained sequence alignment *Given two strings* S_1 *and* S_2 *and a constraint* $\mathcal{P} = \langle P_1, \ldots, P_m \rangle$, *along with* m *"correct" intervals* $[c_1, c'_1], \ldots, [c_l, c'_l]$ *of occurrences (one for each pattern) in* S_1 *for each* P_l *such that the non-overlapping condition is satisfied. Find an alignment of* S_1 *and* S_2 *satisfying* $\mathcal{P}$ *such that the score is minimized.*

The following is the original problem studied in [14, 3, 16] (in fact generalized here). In Sec. 4, a more efficient approximation algorithm over that in [3] is proposed.

Constrained multiple sequence alignment (CMSA) *Given a set of sequences and a constraint* $\mathcal{P}$, *find an alignment satisfying* $\mathcal{P}$ *with the minimum score.*

Since the multiple sequence alignment problem is NP-hard [17, 2], it is clear that CMSA is also NP-hard. Tang et al. [14] and Tsai et al. [16] solved this problem by a progressive method, and Chin et al. [3] adopted an approximation algorithm. The method proposed in Sec. 3 improves Chin et al.'s 2-approximaiton algorithm.

3 The algorithm

In this section, the algorithm for the one-annotated version is developed. Let $\mathcal{P} = \langle P_1, \ldots, P_m \rangle$ be a motif constraint. Suppose that we are given S_1 and S_2 and want to find the optimal alignment satisfying $\mathcal{P}$. Assume that the correct occurrence positions in S_1 is given. First we find on S_2 the occurrence positions for the patterns in $\mathcal{P}$. This is discussed in Sec. 3.1 and Sec. 3.2. In Sec. 3.3, the main algorithm is presented.

3.1 Annotation of type-1 patterns

For each pattern P_l in the constraint, we use $|\Sigma|$ bits to represent $P_l(i)$. For example, if $\Sigma = \{\mathrm{A}, \mathrm{T}, \mathrm{C}, \mathrm{G}\}$ (the order is relevant) and $P_l(i) = \{\mathrm{A}, \mathrm{C}\}$, then $P_l(i)$ is represented as $(1, 0, 1, 0)$. These bits can be stored compactly in computer words to enhance practical performance. Hence all the patterns can be converted into this form in $O(L|\Sigma|)$ time and space, where $L = \sum_{i=1}^{m} |P_i|$. It is then easy to determine all positions of occurrences for all patterns by sliding window (and simple arithmetic and bit operations) in $O(Ln)$ time. The overall space needed is $O(L|\Sigma| + r)$, where r is the total number of occurrences of all patterns in S_2. Let $j_1 \le \cdots \le j_{c(S_2;P_l)}$ be the positions of occurrences of P_l in S_2. Let $start_x(S_2; P_l) = j_x$. Also define $end_x(S_2; P_l) = j_x + |P_l| - 1$. We keep the list $occur(S_2; P_l) = \langle [start_1(S_2; P_l), end_1(S_2; P_l)], \ldots, [start_{c(S_2;P_l)}(S_2; P_l), end_{c(S_2;P_l)}(S_2; P_l)] \rangle$, where $c(S_2; P_l)$ is the number of occurrences of P_l in S_2.

Since each type-1 pattern is in fact a set of strings, one may want to apply suffix tree technique or Aho-Corasick algorithm (see, e.g., [7]) to find positions of occurrences. But it turns out to be uneconomical to enumerate all possible strings in P_l, which is needed for a direct application of these two methods. For an extreme example, if $P_l = \Sigma^{|P_l|}$, then the number of strings in P_l is $|\Sigma|^{|P_l|}$, which grows exponentially in $|P_l|$. On the other hand, it is a restricted regular expression, and the use of general regular expression matching algorithms [7] is not necessary.

3.2 Annotation of type-2 patterns

First of all, we give the following lemma that helps to ensure some property of the occurrence list. It is mentioned in [13] using the notion of grid graph. Define $f(x; X) = \min\{i : x - \#spaces(i; X) = x\}$ for a string X and index x.

Lemma 1 *Let (X', Y') be an alignment (not necessarily optimal) of X and Y, and let (X'', Y'') be an alignment of X and $Y[i..j]$. Then there exists a pair of indices (x, y) such that (X', Y') can be decomposed as $(X'[1..f(x;X')]X'[f(x;X')+1..|X'|], Y'[1..f(y;Y')]Y'[f(y;Y')+1..|Y'|])$, and (X'', Y'') can be decomposed as $(X''[1..f(x;X'')]X''[f(x;X'')+1..|X''|], Y''[1..f(y;Y'')]Y''[f(y;Y'')+1..|Y''|])$.*

For type-2 patterns, we use simple dynamic programming to find the occurring intervals. For each pattern P_l, $1 \le l \le m$, we compute $D(i,j)$, the smallest weighted edit distance d_E between $P_l[1..i]$ and all suffixes of $S_2[1..j]$. Let $S_2[j'..j]$ be such a suffix. Then define $src(i,j) = j'$. If $D(i,j) = d_E(P_l[1..i], S_2[j'..j]) = d_E(P_l[1..i], S_2[j''..j])$, then let $src(i,j) = \max(j', j'')$ (or other tie-breaking rule). After the computation of all entries of $D(i,j), 1 \le i \le |P_l|, 1 \le j \le |S_2|$, we keep all intervals $[src(|P_l|,j), j]$ such that $d_E(P_l, S_2[src(|P_l|,j)..j]) = D(|P_l|,j) \le t$ in list $occur(S_2; P_l)$, where t is the threshold value specified by the user, and $c(S_2; P_l)$ is the number of j such that $D(|P_l|,j) \le t$. By Lemma 1 and the tie-breaking rule, if $end_x > end_y$, then $start_x \ge start_y$. Note that here we are finding occurrence intervals, so we concern only about edit distance (the score), not how P_l aligns with S_2. This processing takes $O(Ln)$ time and $O(n + r)$ space, where $r = \sum_{l=1}^{m} c(S_2; P_l)$.

3.3 The main algorithm

Before the presentation of the algorithm, it should be noted that the problem can be solved by the $O(mn^2)$ time and space algorithm in [3] (with some modifications). However, it is natural to ask whether the additional information given in this version (i.e., the positions of occurrences in one of the strings) can be utilized to improve the time and space bounds. The answer turns out to be positive, and is explored in this section.

To find the optimal constrained alignment of S_1 and S_2, we compute table T using the following recurrences:

1. $T[0,0] = 0$. $T[0,j] = T[0,j-1] + \delta(-, S_1[j])$ if $0 < j < start(S_1, P_1)$. $T[i,0] = T[i-1,0] + \delta(S_2[i], -)$ if $i > 0$.
2. If $j \notin [start(S_1; P_l), end(S_1; P_l)]$ for all l, then $T[i,j] = \min\{T[i-1,j] + \delta(S_2[i], -), T[i-1,j-1] + \delta(S_2[i], S_1[j]), T[i,j-1] + \delta(-, S_1[j])\}$.
3. If $j = start(S_1; P_l)$ for some l,
$$T[i,j] = \begin{cases} T[i-1,j-1] & \text{if } i = start_x(S_2; P_l) \text{ for some } 1 \le x \le c(S_2; P_l); \\ \texttt{null} & \text{otherwise.} \end{cases}$$
4. If $j = end(S_1; P_l)$ for some l,
$$T[i,j] = \begin{cases} \infty & \text{if } i = 0; \\ \min \begin{cases} T[start_x(S_2; P_l), start(S_1; P_l)] + cost(P_l) \\ T[i-1,j] + \delta(S_2[i], -) \end{cases} & \text{if } i = end_x(S_2; P_l); \\ T[i-1,j] + \delta(S_2[i], -) & \text{otherwise.} \end{cases}$$
5. If $j \in (start(S_1; P_l), end(S_1; P_l))$ for some l, then $T[i,j] = \texttt{null}$ (it can be left undefined).

Observe that T has a property that if $j \in [end(S_1; P_l), start(S_1; P_{l+1}))$ for some $l < m$, or if $j \ge end(S_1; P_l)$ for $l = m$, then $T[i,j]$ is the minimum score of aligning $S_2[1..i]$ and $S_1[1..j]$ satisfying the (sub-)constraint $\mathcal{P}_l$. It follows that $T[|S_2|, |S_1|]$ is the optimal score of constrained alignment of S_1 and S_2 satisfying the constraint $\mathcal{P}$.

In the above recurrences, it may seem that to determine to which interval j belongs, it takes time more than just a constant. It turns out to be unnecessary; if the computation proceeds either row-wise or column-wise, then by the sorted-property of the list $occur(S_2; P_l)$, each such determination takes only $O(1)$ time. Also note that the annotation process yields no ambiguity for the determination of $start_x(S_2; P_l)$ given $end_x(S_2; P_l)$.

With the above recurrences, we can now easily compute the optimal constrained alignment of S_1 and S_2. It is clear that $O(n^2)$ time suffices. To reduce space requirements to $O(n)$, a little care is needed. First, it is easy to see that if we are only to compute the score of the optimal constrained alignment, instead of the alignment itself, then the above recurrences can be applied and takes only $O(n)$ space. Besides, the columns in T that satisfy the condition in 5 are negligible; this simplifies the situation one faces. If some care is taken, Hirchsberg's well-known divide-and-conquer technique [8] can be modified to apply here (divide the columns). This reduces the required space to $O(n)$.

Recall that the occurrence-finding phase takes $O(Ln)$ time and $O(L|\Sigma| + r)$ space for type-1 pat-

terns, and $O(Ln)$ time and $O(n+r)$ space for type-2 patterns, where L is the total length of the patterns in the constraint and r is the total number of occurrences of all patterns in S_2. Hence it takes $O(n^2 + Ln)$ time and $O(n + L|\Sigma|)$ space for type-1 constraints, and the same time and $O(n + r)$ space for type-2 constraints. Let's take a closer look at these bounds. First, $L = O(n)$ by the non-overlapping condition. Also, it is common to assume that $|\Sigma|$ is a constant. In addition, it is clear that r is bounded above by $O(mn)$. Hence the time and space needed are essentially $O(n^2)$ and $O(n+r)$, respectively. Note that if constraints are defined as in [3], $r = O(n)$ and the space complexity is $O(n)$.

To incorporate the analysis for important regions that are considered not necessary, a simple weighting scheme can be used, as in [4]. Specifically, for $x, y \in \Sigma \cup \{-\}$, $\delta(x, y)$ can be substituted by $\delta(x, y) \times w_1(x) \times w_2(y)$, where w_1 and w_2 are weights associated with the importance of characters in the first and second string, respectively.

4 A more efficient approximation algorithm for CMSA

Chin et al. [3] extended Gusfield's well-known center-star approximation algorithm for the multiple sequence alignment problem [6] to deal with CMSA, giving a $(2 - \frac{2}{k})$ approximation ratio. The approximation ratio is guaranteed if δ satisfies triangle inequality [6, 3]. It turns out that the algorithm developed in the previous section improves the performance of the 2-approximation algorithm in [3], although the definition of patterns is more general here. We now explore this.

Suppose we are given k strings $S_1, \ldots, S_k$, along with a constraint $\mathcal{P}$. Note that the occurrence intervals are not given for this problem. For convenience, let $n = \max\{|S_1|, \ldots, |S_k|\}$. The modified algorithm is outlined below.

1. **Occurrence-finding phase** Find all intervals of occurrences of all patterns P_l on all strings.

2. **Best center sequence selection phase** Looping over all k sequences, consider each the center sequence once. Suppose that S_i is now the center sequence. Then for each possible list of occurrence intervals $[start_{c_1}(S_i; P_1), end_{c_1}(S_i; P_1)], \ldots, [start_{c_m}(S_i; P_m), end_{c_m}(S_i; P_m)]$ ($(c_1, \ldots, c_m)$ is referred to as "constrained list" later) of patterns $P_1, \ldots, P_m$ on S_i, $1 \le c_l \le c(S_i; P_l)$, S_i is pair-wise aligned with all strings in $\{S_1, \ldots, S_k\} \setminus S_i$ with respect to this constrained list. The sum of these $k-1$ pair-wise scores is kept for later comparison. Note that this is exactly the case when the "correct" occurrence intervals on S_i are given; the alignment can be done by the algorithm in the previous section.

   We then determine for what choices of S_i and constrained list, the sum of pair-wise scores is minimized. Such S_i and $(c_1, \ldots, c_m)$ is used in the following step. This S_i is referred to as the best center sequence and $(c_1, \ldots, c_m)$ as the best constrained list.

3. **Merging phase** The $k-1$ pair-wise alignments of the best center sequence to all other sequences are merged, under the best constrained list, in the same way as in [3].

The first phase takes $O(kLn)$ time and $O(L|\Sigma| + R)$ space, where R is the total number of occurrences of all patterns in all strings. It is clear that R is bounded by $O(kmn)$. The second phase takes $O(Ck^2n^2)$ time and $O(n + r)$ additional space, since in this phase we concern only about the scores of alignments, and we need not store the alignment results. C is the maximum number of valid constrained lists in a single input string; the additional $O(r)$ space is due to the enumeration of valid constrained lists for all sequences, where $r = \max_{1 \le i \le k}\{\sum_{l=1}^{m} c(S_i; P_l)\}$. The third phase takes $O(kn^2)$ time and $O(kn)$ additional space. Hence the total time and space taken are $O(Ck^2n^2)$ and $O(kn + R)$, respectively. In particular, if the definition for patterns in [3] is adopted, our algorithm takes $O(Ck^2n^2)$ time and $O(kn)$ space, as opposed to $O(Cmk^2n^2)$ time and $O(mk^2n^2)$ space for the algorithm in [3].

5 Conclusions

In this paper, we consider a case of pair-wise constrained sequence alignment, where occurrences of patterns in one of the string are known. This is useful when determining whether a query sequence has some properties known or hypothesized by biologists, so that it could have some function. We propose an algorithm taking $O(n^2)$ time and $O(n+r)$ space, where r is the number of occurrences of the patterns in the constraint. The analysis admitted here is thus more general than

previous constrained sequence alignment results [14, 3, 16]. Also, the algorithm improves previous approximation algorithm for CMSA [3], whose effectiveness had been demonstrated in [3], both in generality and efficiency, allowing one to conduct more practical and larger scale analysis.

We are currently developing efficient algortihms that can incorporate more kinds of constraints into the alignment. The first we would consider is arc annotation, which is useful for representating RNA secondary structures. Arc annotations and motif constraints together make the prediction and analysis of TIM barrel structure more plausible, since such structure contains alpha helices and beta sheets. On the other hand, we have completed an algorithm that can incorporate the information about ranges between patterns. It is hoped that combining all these factors, the analysis will be more realistic and flexible for biologists.

References

[1] Abendroth, J., Niefind, K., Schomburg, D.: X-ray Structure of a Dihydropyrimidinase from Thermus sp. at 1.3Å Resolution. J. Mol. Biol. **320** (2002) 143–156

[2] Bonizzoni, P., Vedova, G. D.: The Complexity of Multiple Sequence Alignment with SP-score that is a metric. Theoret. Comput. Sci. **259** (2001) 63–79

[3] Chin, F. Y. L., Ho, N. L., Lam, T. W., Wong, P. W. H., Chan, M. Y.: Efficient Constrained Sequence Alignment with Performance Guarantee. In: Proc. Comput. Syst. Bioinfo. (CSB'03). IEEE (2003) 337–346

[4] Evans, P. A.: Algorithms and Complexity for Annotated Sequence Analysis. Ph. D Thesis, Department of Computer Science, University of Victoria, Canada (1999)

[5] Faisst, S., Meyer, S.: Compilation of Vertebrate-encoded Transcription Factors. Nucleic Acids Research **20** (1992) 3–26

[6] Gusfield, D.: Efficient Methods for Multiple Sequence Alignment with Guaranteed Error Bounds. Bul. Math. Biol. **30** (1993) 141–154

[7] Gusfield, D.: Algorithms on Strings, Trees, and Sequences. Cambridge University Press (1997)

[8] Hirschberg, D. S.: A Linear Space Algorithm for Computing Maximal Common Subsequences. Comm. ACM. **18** (1975) 341–343

[9] Jiang, T., Lin, G., Ma, B., Zhang, K.: A General Edit Distance between RNA Structures. J. Comput. Biol. **9** (2002) 371–388

[10] Jiang, T., Xu, Y., Zhang, M. Q. (ed.): Current Topics in Computational Molecular Biology. The MIT Press (2002)

[11] Katoh, K., Misawa, K., Kuma, K.-I., Miyata, T.: MAFFT: A Novel Method for Rapid Multiple Sequence Alignment Based on Fast Fourier Transform. Nucleic Acids Res. **30** (2002) 3059–3066

[12] Lin, G.-H., Chen, Z.-Z., Jiang, T., Wen, J.: The Longest Common Subsequence Problem for Sequences with Nested Arc Annotations. J. Comput. Syst. Sci. **65** (2002) 465–480

[13] Schmidt, J. P.: All Highest Scoring Paths in Weighted Grid Graphs and Their Application to Finding All Approximate Repeats in Strings. SIAM J. Comput. **27** (1998) 972–992

[14] Tang, C. Y., Lu, C. L., Chang, M. D. T., Sun, Y. J., Tsai, Y. T., Chang, J. M., Chiou, Y. H., Wu, C. M., Chang, H. T., Chou, W. I., Chiang, S. C.: Constrained Sequence Alignment Tool Development and its Application to RNase Family Alignment. J. Bioinfo. Comput. Biol. **1** (2003) 267–287

[15] Taylor, W. R.: Motif-biased Protein Sequence Alignment. J. Comput. Biol. **1** (1994) 297–310

[16] Tsai, Y. T., Lu, C. L., Yu, C. T., Huang, Y. P.: MuSiC: A Tool for Multiple Sequence Alignment with Constraints. Bioinformatics (to appear)

[17] Wang, L., Jiang, T.: On the Complexity of Multiple Sequence Alignment. J. Comput. Biol. **1** (1994) 337–348